

سوالات تحلیلی

سوال ۱: مشخصات یک سیگنال PWM چیست؟

PWM signal characteristics

- *Modulation frequency*: how many pulses occur per second (fixed)
- *Period*: $1/(\text{modulation frequency})$
- *On-time*: amount of time that each pulse is on (asserted)
- *Duty-cycle*: on-time/period
- Adjust on-time (hence duty cycle) to represent the analog value

(مدولاسیون) فرکانس: چند پالس بر ثانیه اتفاق می افتد

پریود/دوره: معکوس (مدولاسیون) فرکانس/دوره یک پالس

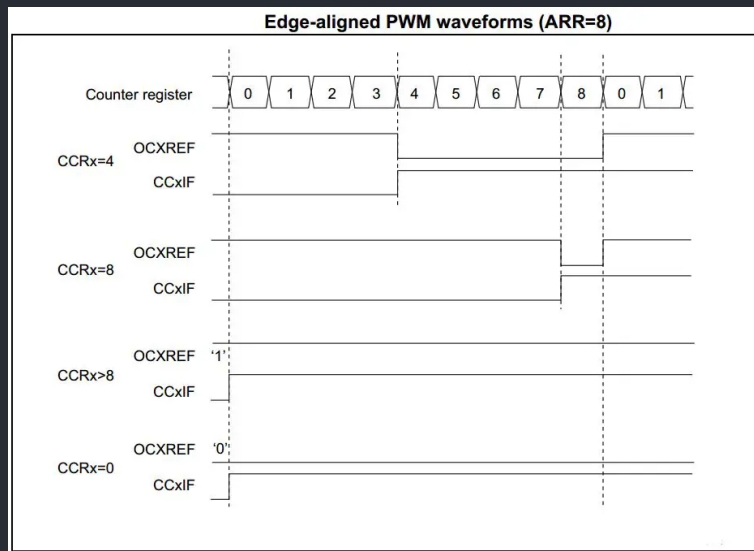
آن تایم/مدت زمان فعال بودن: مدت زمانی که پالس on/فعال است (صفر نیست)

دیوتی سیکل: درصد زمانی که پالس on/فعال است

سوال ۲: تفاوت مد یک و دو PWM در STM32F4 چیست؟

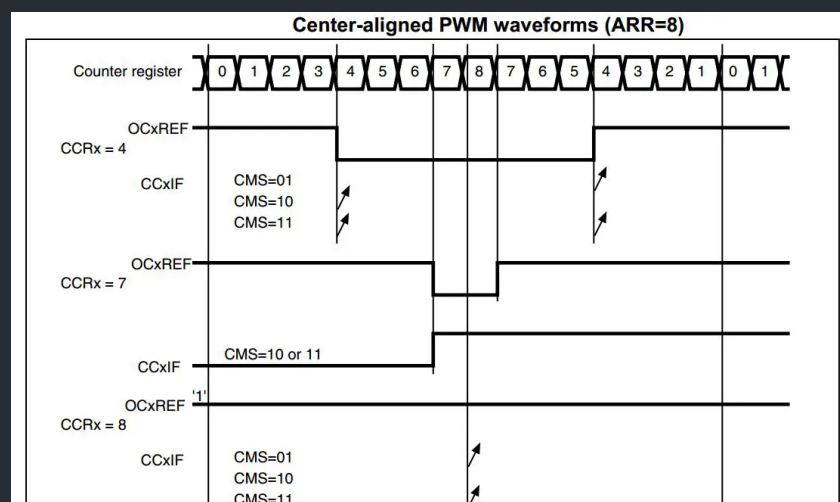
Mode1: Edge-Aligned Mode

این مود خود دارای دو کانفیک Up-Counting و Down-Counting است که با رجیستر TIMx_CR1 قابل تنظیم است. با توجه به کانفیک، سیگنال PWM ما (OCxREF) تا زمانی فعال است که به ترتیب $TIMx_CNT < TIMx_CCRx$ و $TIMx_CNT > TIMx_CCRx$ باشد.



Mode2: Center-Aligned Mode

در این مود، compare flag زمانی ست می‌شود که counter ما یک بار پشت هم count-up و count-down (و یا بسته به کانفیک آن جابجا) کند.



سوال ۳: رجیسترهای کانالهای capture / compare را نام ببرید و ذکر کنید در مدهای مختلف چه کاربردی دارند.

TIMx_CCMR1/2: Capture/Compare Mode Register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]			IC1PSC[1:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- **CCxS** (Capture/Compare Selection): Input Capture mode
 - **CC1S[1:0]** in CCMR1:
 - 00: The channel is configured as input, mapped on TI1 (PA0 pin).
 - 01: The channel is configured as input, mapped on TI2 (PA1 pin).
 - 10: The channel is configured as input, mapped on TI3 (PA2 pin).
 - 11: The channel is configured as input, mapped on TI4 (PA3 pin).
 - **CC2S[1:0]** in CCMR1:
 - 00: The channel is configured as input, mapped on TI2 (PA1 pin).
 - 01: The channel is configured as input, mapped on TI1 (PA0 pin).
 - 10: The channel is configured as input, mapped on TI4 (PA3 pin).
 - 11: The channel is configured as input, mapped on TI3 (PA2 pin).
 - **ICxPSC** (Input Capture Prescaler): determines the frequency at which input captures are performed.
 - **ICxF** (Input Capture Filter)
-
- **CCxS** (Capture/Compare Selection): Output Compare mode
 - **CC1S[1:0]** CCMR1:
 - 00: The channel is configured as output, PWM mode 1.
 - 01: The channel is configured as output, PWM mode 2.
 - 10: The channel is configured as output, PWM mode 3.
 - 11: The channel is configured as output, PWM mode 4.
 - **CC2S[1:0]** CCMR1:
 - 00: The channel is configured as output, PWM mode 1.
 - 01: The channel is configured as output, PWM mode 2.
 - 10: The channel is configured as output, PWM mode 3.
 - 11: The channel is configured as output, PWM mode 4.
 - **OCxFE** (Output Compare Fast Enable): In this mode, the output compare signal can change more quickly, but with a reduced resolution.

- **OCxPE** (Output Compare Preload Enable): When enabled, the compare value is loaded from a preload register, allowing for atomic updates and synchronization with the timer's counter.
- **OCxCE** (Output Compare Clear Enable)
- **OCxM[2:0]** (Output Compare Mode)
 - 000: Frozen
 - 001: Set on match
 - 010: Clear on match
 - 011: Toggle on match
 - 100: Force inactive
 - 101: Force active
 - 110: PWM mode 1: clear on overflow
 - 111: PWM mode 2: set on overflow

TIMx_CCER: Capture/Compare Enable Register

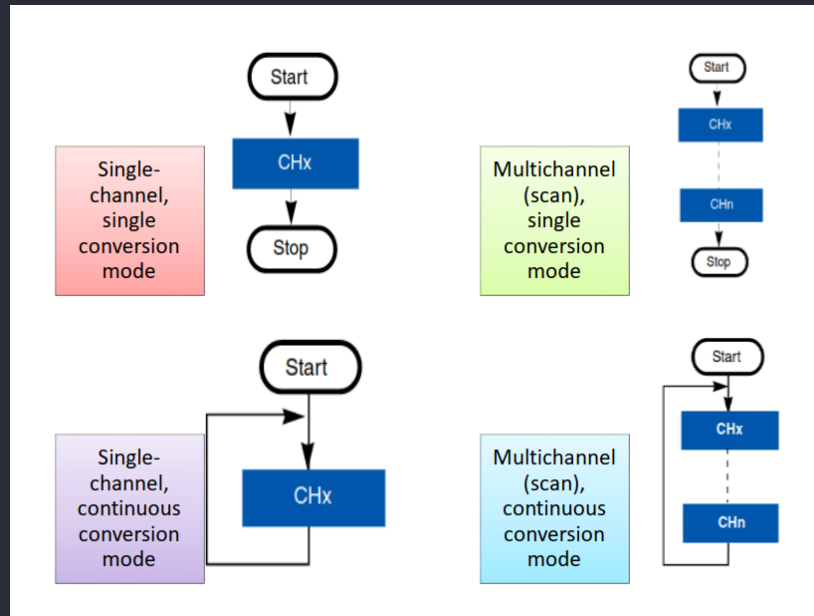
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

- **CCxE:** enables the capture
- **CCxNP/CCxP:** determine to be sensitive to falling/rising edge

TIMx_CCR: Capture/Compare Register

سوال ۴: مُدهای کاری مختلف تبدیل آنالوگ به دیجیتال در میکروکنترلرهای STM32 را چیست و چه تفاوتی با هم دارند؟

دارای 4 مود است:



1) Single-channel, single conversion mode:

در این مود، کانورت به طور همزمان تنها یک کانال ورودی آنالوگ را کانورت می‌کند و هر بار منتظر می‌ماند تا trigger شود و سپس کار کانورت کردن خود را آغاز کند.

2) Multichannel (scan), single conversion mode:

در این مود کانورت به طور همزمان چندین (تا 16) کانال را کانورت می‌کند و مانند مود قبلی هر بار باید trigger شود.

3) Single-channel continuous conversion mode:

در این مود کانورت تنها یک کانال را کانورت می‌کند، با این تفاوت که نیاز به trigger شدن ندارد و بلافاصله بعد از کانورت شدن سیگنال آنالوگ فعلی، به سراغ سیگنال بعدی می‌رود.

4) Multichannel (scan) continuous conversion mode:

در این مود کانورت به طور همزمان چندین (تا 16) کانال را کانورت می‌کند و همانند مود قبلی نیاز به trigger شدن ندارد و پس از کانورت شدن سیگنال های فعلی، به سراغ سیگنال ها بعدی می‌رود.

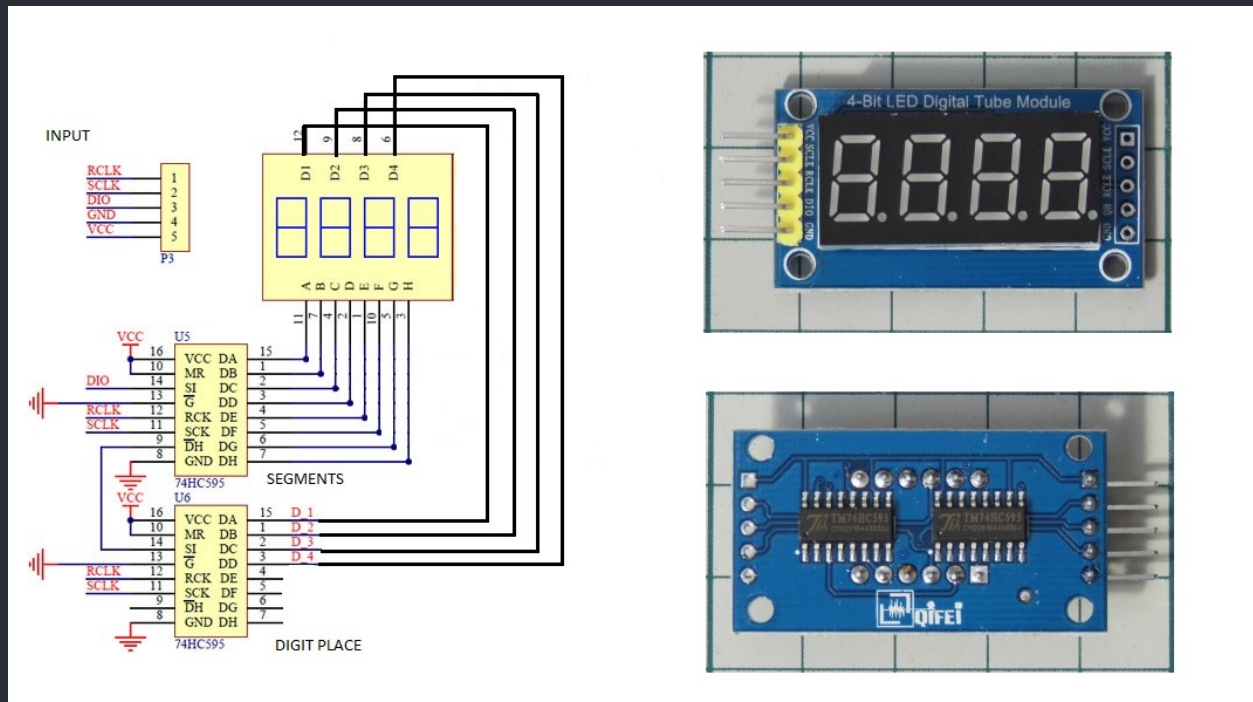
سوال ۵: تراشه 74595 چیست و چگونه کار میکند؟ یک digital tube چیست و چگونه با این تراشه ساخته میشود؟

تراشه 74HC595 یک IC شیفت رجیستر معروف است که کاربرد آن در داده های

- SIPO (Serial input Parallel output)

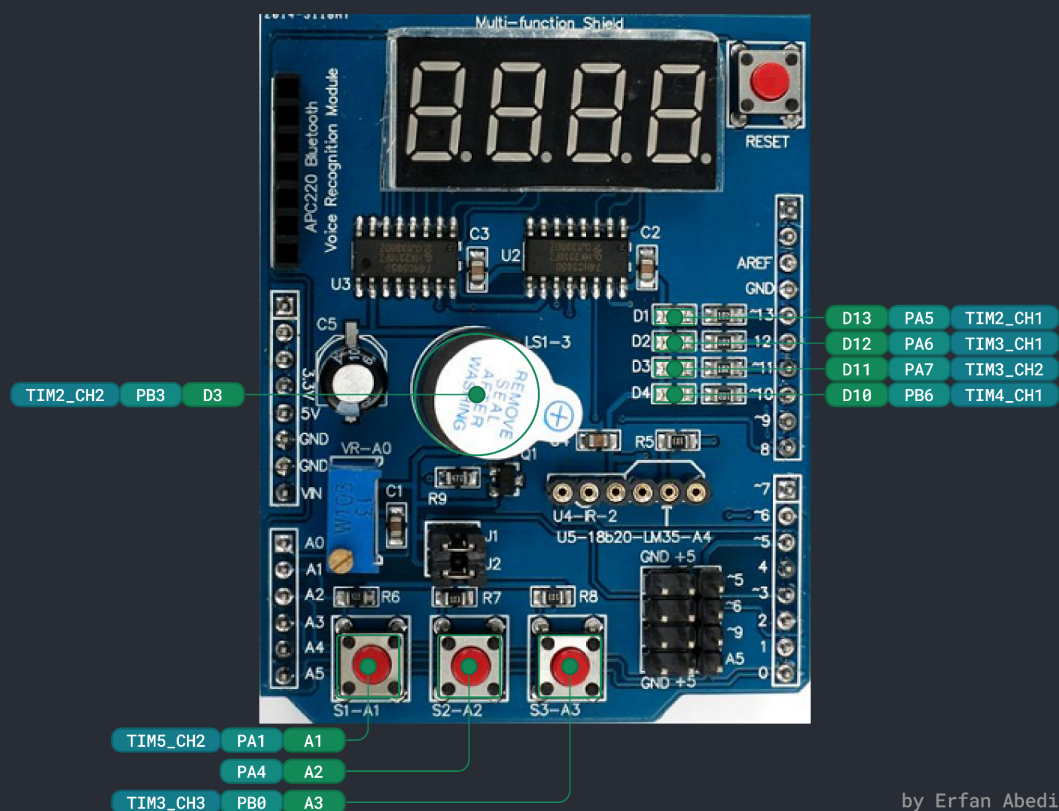
برای کنترل چندین خروجی با کاهش پین های اشغال شده ریزپردازنده ها یا مدارات است.

digital tube یا digital display ها صفحه نمایش هایی اند که برای نمایش چندین عدد یا کاراکتر استفاده میشوند و در ماشین حساب، ساعت های دیجیتالی و ادوات اندازه گیری الکترونیکی به کار میروند.



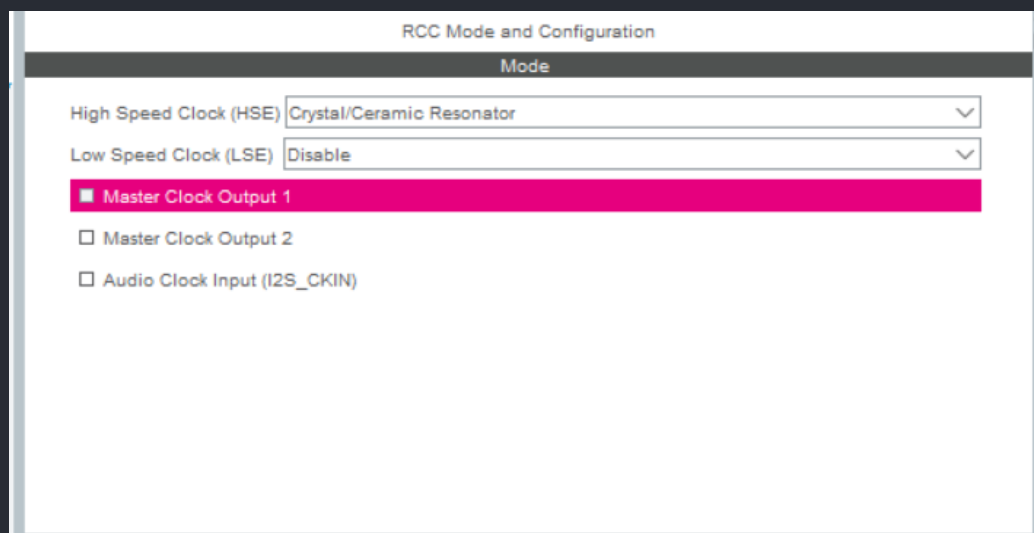
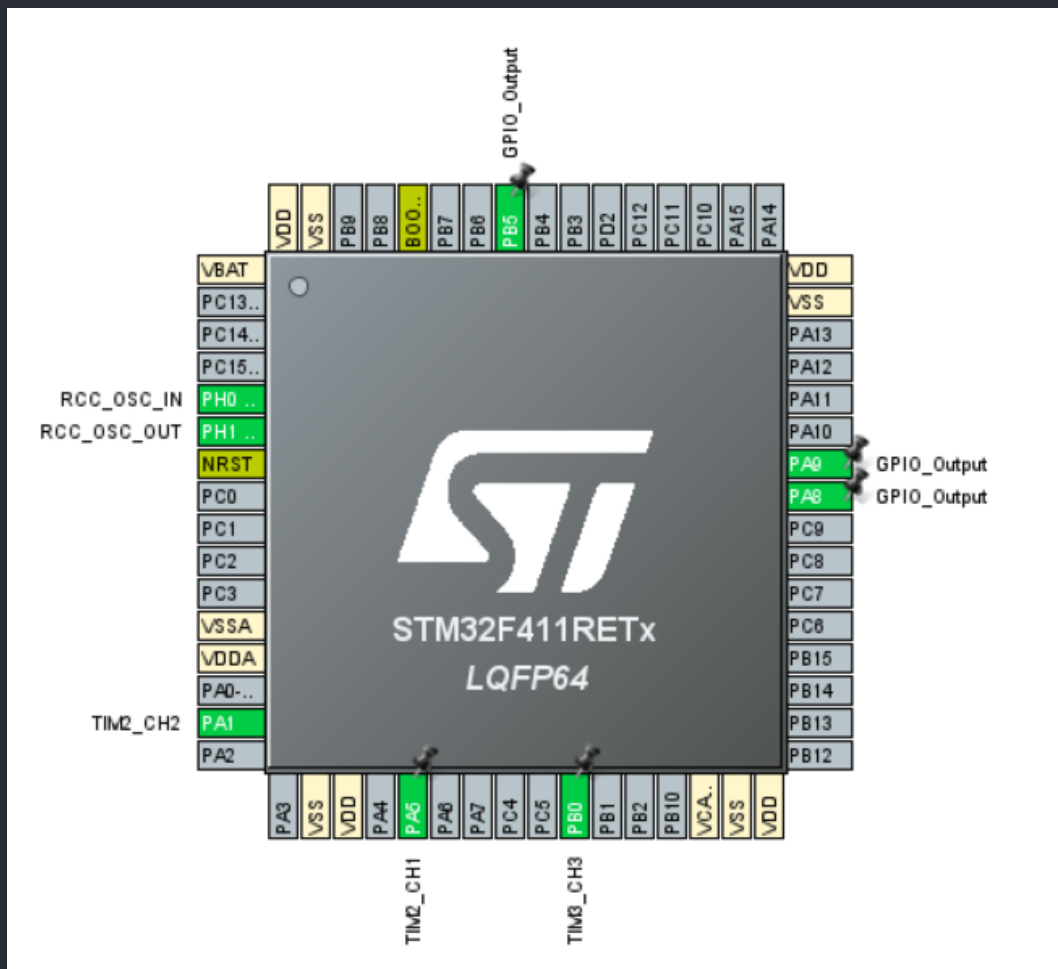
سوال ۶: در این آزمایشگاه از شیلد مولتی فانکشن آردوینو استفاده خواهد شد که ادوات متنوع و امکانات دیجیتال و آنالوگی روی خود دارد. با بررسی شماتیک و اتصالات شیلد، نقشه پینها (pinout) برد Nucleo 64 مورد استفاده در آزمایشگاه و نیز جداول Alternate Function پین ها در بخش ۴ داده برگ (دیتاشیت) میکرو STM32F401 مشخص کنید هر یک از دکمه های فشاری، LED ها و باز روی شیلد به کدام کانالهای کدام تایمرها میتوانند متصل شوند. پاسخ را در یک جدول ذکر کنید.

Components	Arduino Pins	STM32F401 Pins	AF01	AF02
LED1 / D1	D13	PA5	TIM2_CH1/ TIM2_ETR	-
LED2 / D2	D12	PA6	TIM1_BKIN	TIM3_CH1
LED3 / D3	D11	PA7	TIM1_CH1N	TIM3_CH2
LED4 / D4	D10	PB6	-	TIM4_CH1
Button1 / S1	A1	PA1	TIM2_CH2	TIM5_CH2
Button2 / S2	A2	PA4	-	-
Button3 / S3	A3	PB0	TIM1_CH2N	TIM3_CH3
Buzzer / LS1	D3	PB3	TIM2_CH2	-



گزارش دستور کار 8:

CubeMX:



TIM2 Mode and Configuration

Mode

Slave Mode Disable ▼

Trigger Source Disable ▼

Clock Source Internal Clock ▼

Channel1 Input Capture direct mode ▼

Channel2 Input Capture direct mode ▼

Channel3 Disable ▼

Channel4 Disable ▼

Combined Channels Disable ▼

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

✔ Parameter Settings
✔ User Constants
✔ NVIC Settings
✔ DMA Settings
✔ GPIO Settings

Configure the below parameters :

⏪
⏩
i

▼ Counter Settings

Prescaler (PSC - 16 bits value)	16000-1
Counter Mode	Up
Counter Period (AutoReload Register - 32 bit...)	0xFFFFFFFF-1
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

▼ Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

▼ Input Capture Channel 1

Polarity Selection	Both Edges
IC Selection	Direct
Prescaler Division Ratio	No division
Input Filter (4 bits value)	0

▼ Input Capture Channel 2

Polarity Selection	Both Edges
IC Selection	Direct
Prescaler Division Ratio	No division
Input Filter (4 bits value)	0

صفحه 9 از 30

TIM3 Mode and Configuration

Mode

Slave Mode ▼

Trigger Source ▼

Clock Source ▼

Channel1 ▼

Channel2 ▼

Channel3 ▼

Channel4 ▼

Combined Channels ▼

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

✔ Parameter Settings
✔ User Constants
✔ NVIC Settings
✔ DMA Settings
✔ GPIO Settings

Configure the below parameters :

⏪
⏩
i

▼ Counter Settings

Prescaler (PSC - 16 bits value)	16000-1
Counter Mode	Up
Counter Period (AutoReload Register - 16 bit...)	0xFFFF-1
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

▼ Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

▼ Input Capture Channel 3

Polarity Selection	Both Edges
IC Selection	Direct
Prescaler Division Ratio	No division
Input Filter (4 bits value)	0

TIM4 Mode and Configuration

Mode

Slave Mode Disable ▼

Trigger Source Disable ▼

☒ Internal Clock

Channel1 Disable ▼

Channel2 Disable ▼

Channel3 Disable ▼

Channel4 Disable ▼

Combined Channels Disable ▼

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

✔ Parameter Settings
✔ User Constants
✔ NVIC Settings
✔ DMA Settings

Configure the below parameters :

⏪
⏩
i

▼

Counter Settings

Prescaler (PSC - 16 bits value) 16-1

Counter Mode Up

Counter Period (AutoReload Register - 16 bit... 100-1

Internal Clock Division (CKD) No Division

auto-reload preload Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Reset (UG bit from TIMx_EGR)

کد:

```

main.c
40
41 /* USER CODE END PM */
42
43 /* Private variables -----*/
44 TIM_HandleTypeDef htim2;
45 TIM_HandleTypeDef htim3;
46 TIM_HandleTypeDef htim4;
47
48 /* USER CODE BEGIN PV */
49 static const unsigned char sevenSegHex[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
50 static const unsigned char digitHex[4] = {0xFE, 0xFD, 0xFB, 0xF7};
51 volatile int N = 7;
52 volatile int M = 5;
53
54 int32_t IC_Val1 = 0;
55 int32_t IC_Val2 = 0;
56 uint32_t Difference = 0;
57 int Is_First_Captured = 0;
58 /* USER CODE END PV */
59
60 /* Private function prototypes -----*/
61 void SystemClock_Config(void);
62 static void MX_GPIO_Init(void);
63 static void MX_TIM2_Init(void);
64 static void MX_TIM3_Init(void);
65 static void MX_TIM4_Init(void);
66 /* USER CODE BEGIN PFP */
67 int pascal(int row, int col);
68 void shiftOut(unsigned char num);
69 /* USER CODE END PFP */
70

```

```

main.c
72 /* USER CODE BEGIN 0 */
73 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
74 {
75     if ((htim->Instance == TIM2 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) || (htim->Instance == TIM2 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
76     {
77         if (Is_First_Captured == 0) // if the first rising edge is not captured
78         {
79             IC_Val1 = HAL_TIM_ReadCapturedValue(htim, ((htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) ? TIM_CHANNEL_2 : TIM_CHANNEL_1)); // read the first value
80             Is_First_Captured = 1; // set the first captured as true
81         }
82
83         else // If the first rising edge is captured, now we will capture the second edge
84         {
85             IC_Val2 = HAL_TIM_ReadCapturedValue(htim, ((htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) ? TIM_CHANNEL_2 : TIM_CHANNEL_1)); // read second value
86
87             if (IC_Val2 > IC_Val1)
88             {
89                 Difference = IC_Val2 - IC_Val1;
90             }
91
92             else if (IC_Val1 > IC_Val2)
93             {
94                 Difference = (((htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) ? PeriodTIM2 : PeriodTIM2) - IC_Val1) + IC_Val2;
95             }
96
97             if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2)
98             {
99                 N += ((Difference < 100) ? +1:-1);
100             }
101             else
102             {
103                 M += ((Difference < 100) ? +1:-1);
104             }
105
106             HAL_TIM_SET_COUNTER(htim, 0); // reset the counter
107             Is_First_Captured = 0; // set it back to false
108         }
109     }
110 }

```

```

main.c
111 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
112 {
113     if (htim->Instance == TIM4)
114     {
115         int pasc = pascal(N, M);
116         int digits[4] = {(pasc / 1000) % 10, (pasc / 100) % 10, (pasc / 10) % 10, pasc % 10};
117
118         for(int i = 0; i < 4; i++)
119         {
120             HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
121             shiftOut(sevenSegHex[digits[i]]);
122             shiftOut(digitHex[i]);
123             HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);
124         }
125     }
126 }
127
128 void shiftOut(unsigned char num)
129 {
130     for(int i = 7; i >= 0; i--)
131     {
132         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
133
134         if ((num >> i) & 0x01) == 0x01)
135             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
136         else
137             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
138
139         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
140     }
141 }
142
143 int pascal(int row, int col)
144 {
145     if (col == 1 || row == col)
146         return 1;
147     else
148         return pascal(row - 1, col) + pascal(row - 1, col - 1);
149 }
150
151 /* USER CODE END 0 */

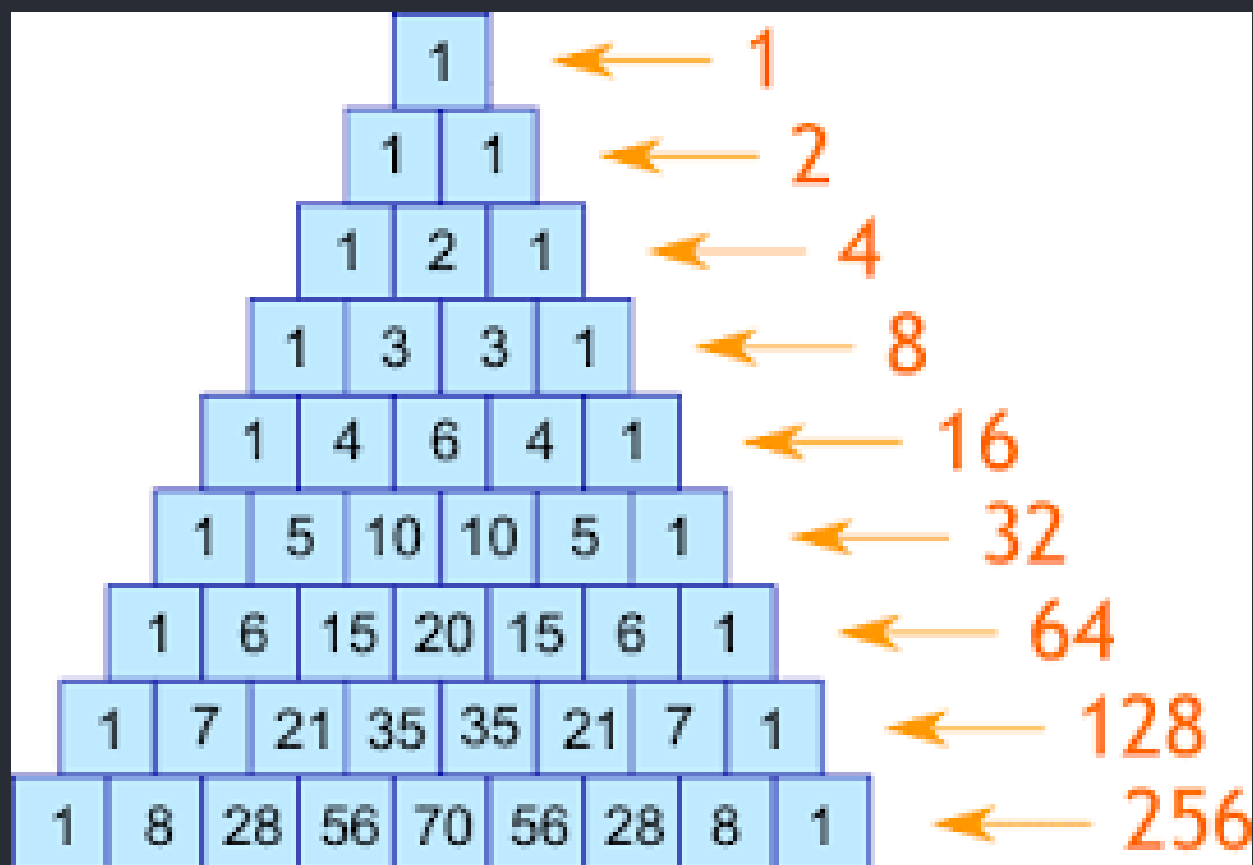
```

```

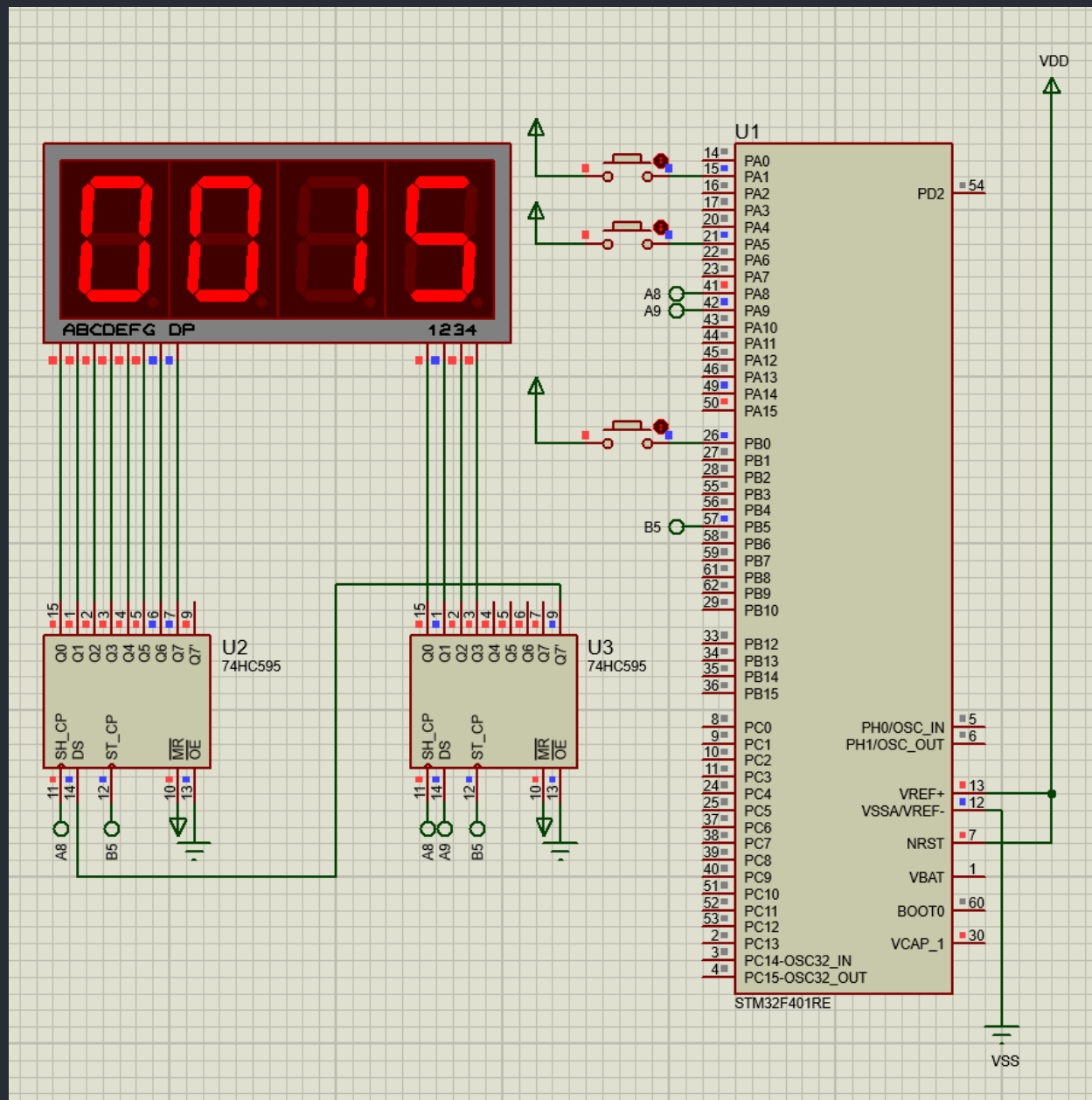
main.c
154 /*
155 int main(void)
156 {
157     /* USER CODE BEGIN 1 */
158
159     /* USER CODE END 1 */
160
161     /* MCU Configuration-----*/
162
163     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
164     HAL_Init();
165
166     /* USER CODE BEGIN Init */
167
168     /* USER CODE END Init */
169
170     /* Configure the system clock */
171     SystemClock_Config();
172
173     /* USER CODE BEGIN SysInit */
174
175     /* USER CODE END SysInit */
176
177     /* Initialize all configured peripherals */
178     MX_GPIO_Init();
179     MX_TIM2_Init();
180     MX_TIM3_Init();
181     MX_TIM4_Init();
182     /* USER CODE BEGIN 2 */
183     HAL_TIM_Base_Start_IT(&htim4);
184     HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1);
185     HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_2);
186     HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_3);
187     /* USER CODE END 2 */
188
189     /* Infinite loop */
190     /* USER CODE BEGIN WHILE */
191     while (1)
192     {

```

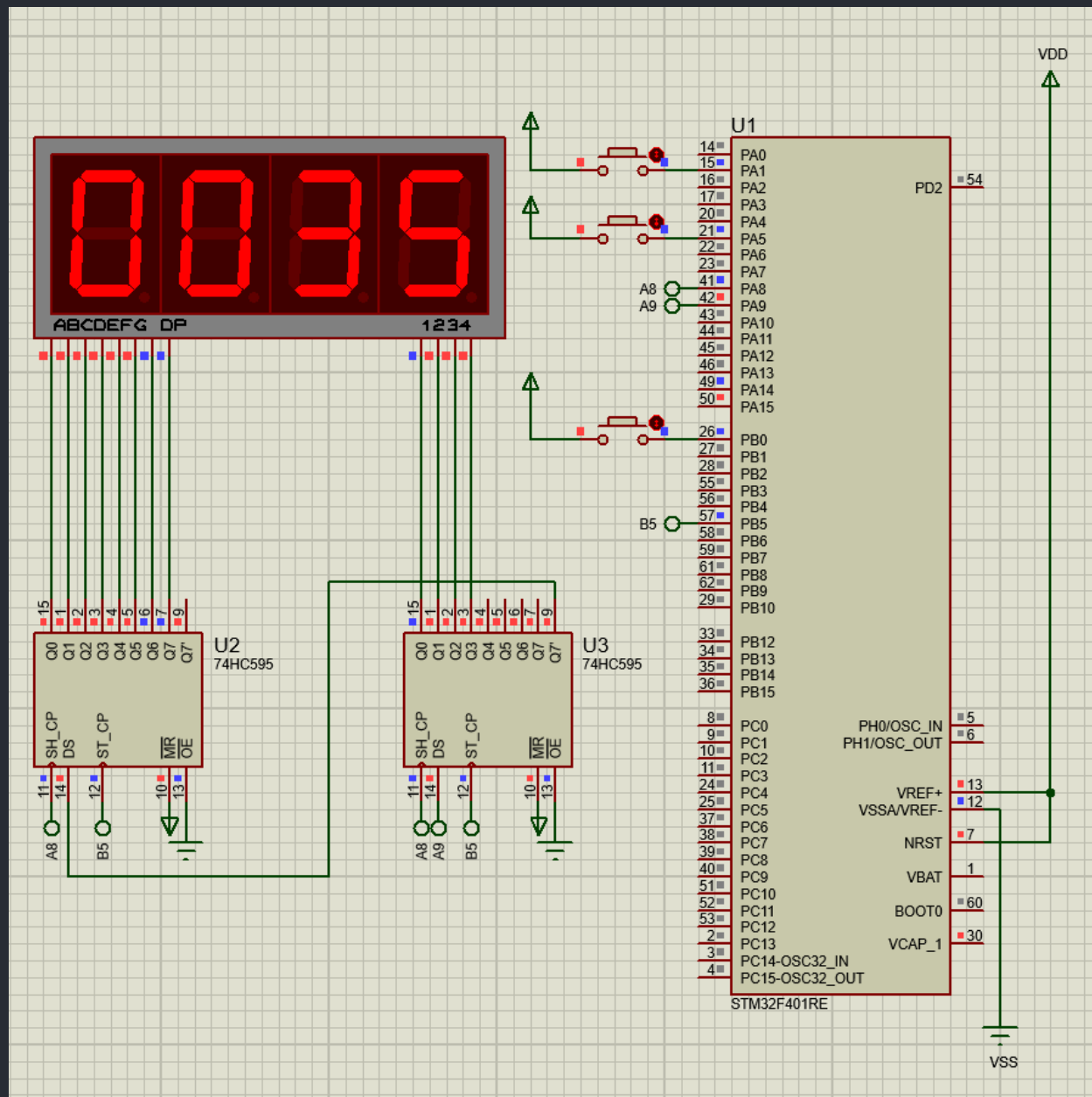
تست:



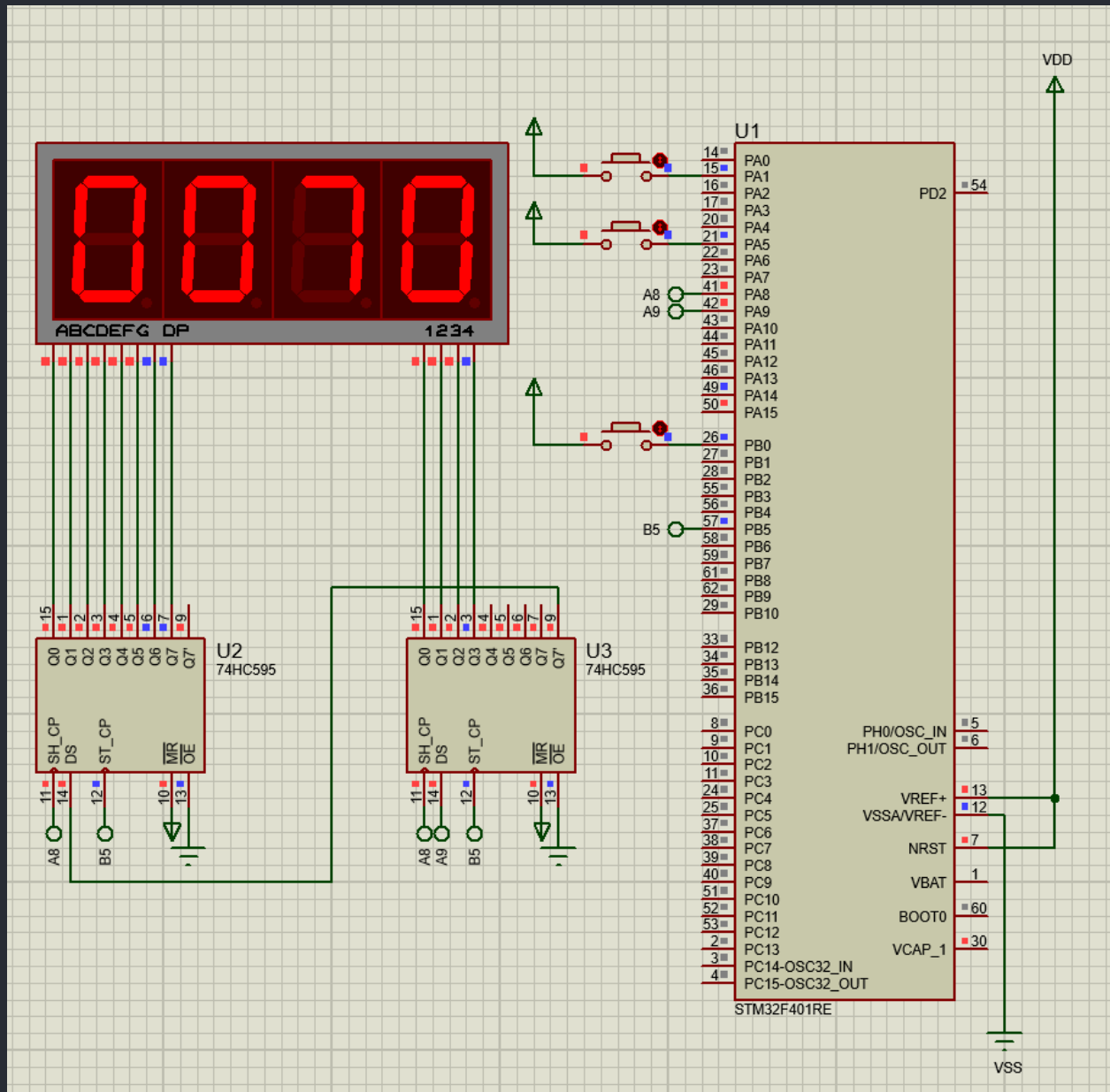
N=7, M=5



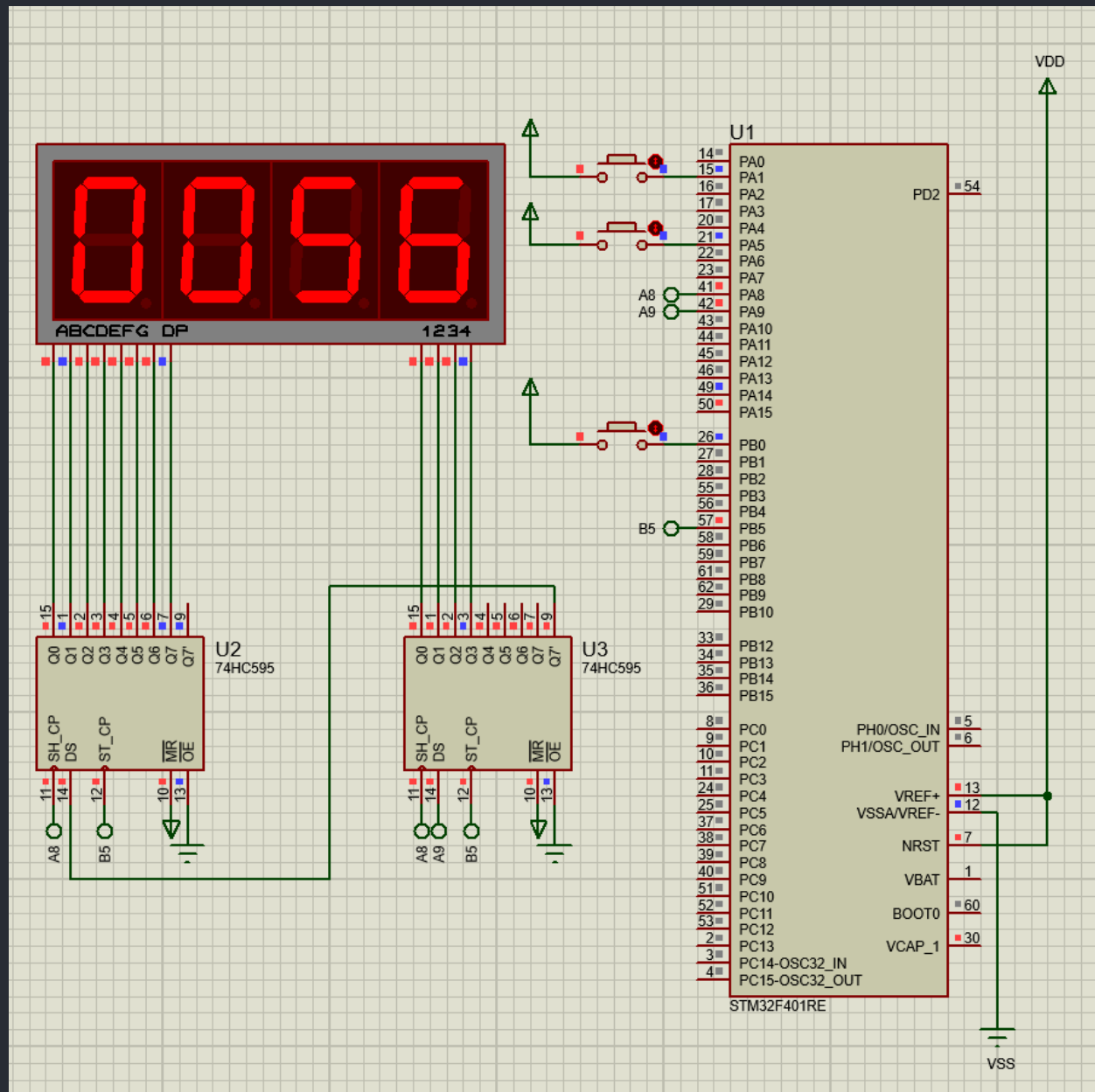
N=8, M=5



N=9, M=5

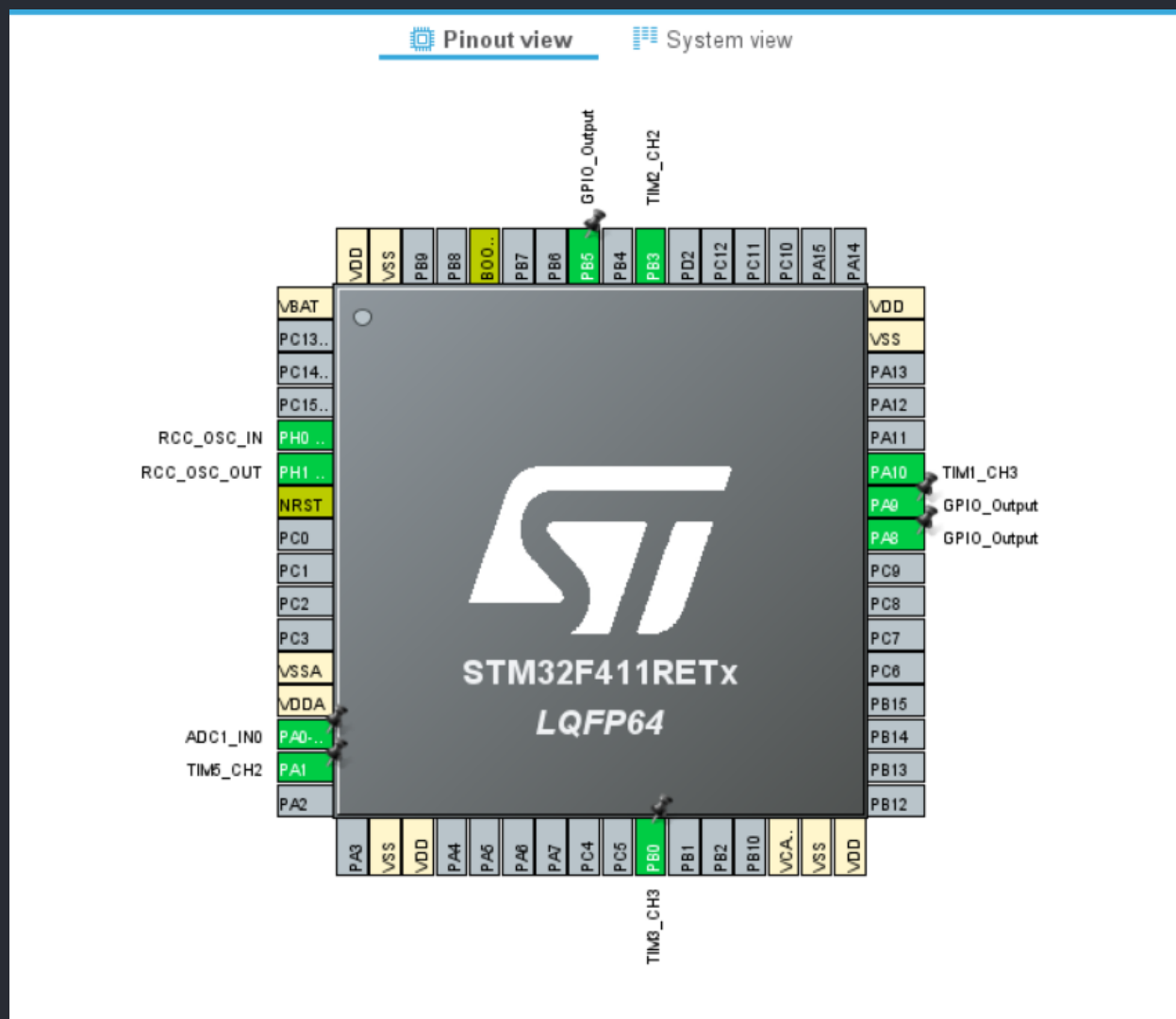


N=9, M=4



گزارش دستور کار 9:

CubeMX:



TIM2 Mode and Configuration

Mode

- Slave Mode: Disable
- Trigger Source: Disable
- Clock Source: Internal Clock
- Channel1: Disable
- Channel2: PWM Generation CH2
- Channel3: Disable
- Channel4: Disable
- Combined Channels: Disable

Configuration

Reset Configuration

☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings
 ☒ Parameter Settings
 ☒ User Constants

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

- Prescaler (PSC - 16 bits value): 1000-1
- Counter Mode: Up
- Counter Period (AutoReload Regi...): 16000-1
- Internal Clock Division (CKD): No Division
- auto-reload preload: Disable

Trigger Output (TRGO) Parameters

- Master/Slave Mode (MSM bit): Disable (Trigger input effect not delayed)
- Trigger Event Selection: Reset (UG bit from TIMx_EGR)

صفحه 21 از 30

ADC1 Mode and Configuration

Mode

- ☒ IN0
- ☐ IN1

Configuration

Reset Configuration

☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

☒ Parameter Settings ☒ User Constants

Configure the below parameters :

Search (Ctrl+F)

- ADC_Settings**
 - Clock Prescaler: PCLK2 divided by 2
 - Resolution: 12 bits (15 ADC Clock cycles)
 - Data Alignment: Right alignment
 - Scan Conversion Mode: Disabled
 - Continuous Conversion Mode: Disabled
 - Discontinuous Conversion Mode: Disabled
 - DMA Continuous Requests: Disabled
 - End Of Conversion Selection: EOC flag at the end of single channel conver...
- ADC_Regular_ConversionMode**
 - Number Of Conversion: 1
 - External Trigger Conversion Source: Timer 1 Capture Compare 3 event
 - External Trigger Conversion Edge: Trigger detection on the rising edge
 - Rank: 1
- ADC_Injected_ConversionMode**
 - Number Of Conversions: 0
- WatchDog**
 - Enable Analog WatchDog Mode: ☐

The screenshot shows the STM32CubeMX Pinout & Configuration window. The left sidebar lists various components, with TIM1 selected. The main area displays the TIM1 Mode and Configuration settings.

Pinout & Configuration

Categories A-Z

- NVIC
- PDM2PCM
- RCC
- RTC
- SDIO
- SPI1
- SPI2
- SPI3
- SPI4
- SPI5
- SYS
- TIM1**
- TIM2
- TIM3
- TIM4
- TIM5
- TIM9
- TIM10
- TIM11
- USART1
- USART2
- USART6
- USB_DEVICE
- USB_HOST
- USB_OTG_FS
- WWDG
- X-CUBE-AI
- X-CUBE-ALGOBUILD
- X-CUBE-ALS
- X-CUBE-AZRTOS-F4
- X-CUBE-PIF1

TIM1 Mode and Configuration

Mode

- Slave Mode: Disable
- Trigger Source: Disable
- Clock Source: Internal Clock
- Channel1: Disable
- Channel2: Disable
- Channel3: Output Compare CH3
- Channel4: Disable
- Combined Channels: Disable
- ☐ Activate-Break-Input
- ☐ Use ETR as Clearing Source

Configuration

Reset Configuration

- ✓ NVIC Settings
- ✓ DMA Settings
- ✓ GPIO Settings
- ✓ **Parameter Settings**
- ✓ User Constants

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

- Prescaler (PSC - 16 bits value): 16000-1
- Counter Mode: Up
- Counter Period (AutoReload Regi...): 100
- Internal Clock Division (CKD): No Division
- Repetition Counter (RCR - 8 bits v...): 0
- auto-reload preload: Disable

Code:

```

107     if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2)
108     {
109         if (!(N == 1 && (Difference >= threshold)))
110             N += ((Difference < threshold) ? +1:-1);
111
112         htim2.Instance->PSC = adc_value/2;
113         HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
114         HAL_Delay((Difference < threshold) ? 200:400);
115         HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_2);
116     }
117     else
118     {
119         if (!(M == N && (Difference < threshold)) || (M == 1 && (Difference >= threshold)))
120             M += ((Difference < threshold) ? +1:-1);
121
122         htim2.Instance->PSC = adc_value;
123         HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
124         HAL_Delay((Difference < threshold) ? 200:400);
125         HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_2);
126     }
127
128     _HAL_TIM_SET_COUNTER(htim, 0); // reset the counter
129     Is_First_Captured = 0; // set it back to false
130 }
131 }
132 }

```

```

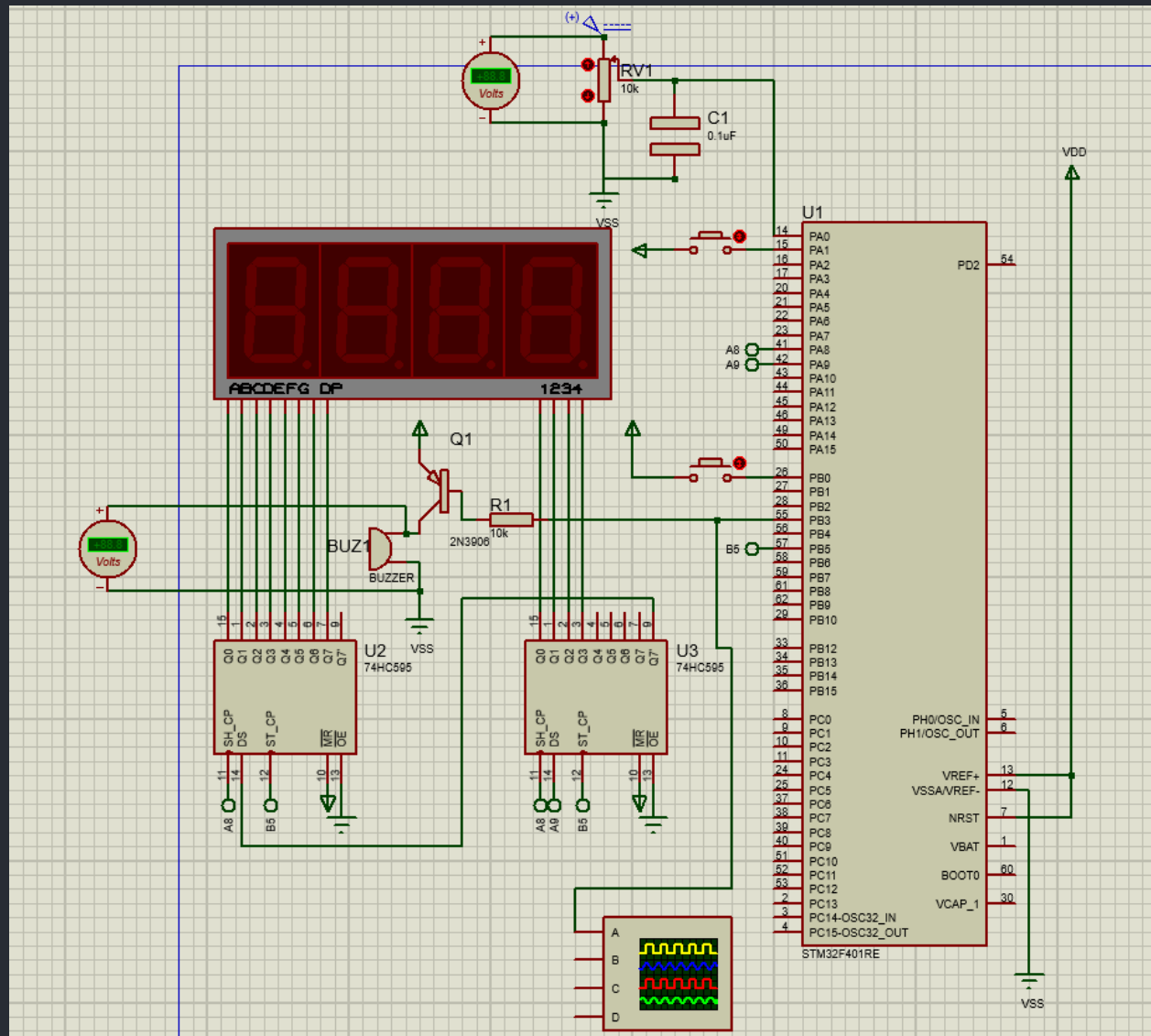
180 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
181 {
182     adc_value = HAL_ADC_GetValue(hadc);
183     HAL_ADC_Start_IT(&hadc1);
184 }
185
186
187 // Read the ADC value
188 uint32_t readADC(void)
189 {
190     // Start the ADC conversion
191     HAL_ADC_Start(&hadc1);
192
193     // Wait for the conversion to complete
194     HAL_ADC_PollForConversion(&hadc1, 1000);
195
196     // Get the converted value
197     uint32_t value = HAL_ADC_GetValue(&hadc1);
198
199     // Return the value
200     return value;
201 }
202

```



```
209 int main(void)
210 {
211     /* USER CODE BEGIN 1 */
212
213     /* USER CODE END 1 */
214
215     /* MCU Configuration-----*/
216
217     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
218     HAL_Init();
219
220     /* USER CODE BEGIN Init */
221
222     /* USER CODE END Init */
223
224     /* Configure the system clock */
225     SystemClock_Config();
226
227     /* USER CODE BEGIN SysInit */
228
229     /* USER CODE END SysInit */
230
231     /* Initialize all configured peripherals */
232     MX_GPIO_Init();
233     MX_TIM3_Init();
234     MX_TIM4_Init();
235     MX_TIM5_Init();
236     MX_TIM2_Init();
237     MX_ADC1_Init();
238     MX_TIM1_Init();
239     /* USER CODE BEGIN 2 */
240     HAL_TIM_Base_Start_IT(&htim4);
241     HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_2);
242     HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_3);
243     HAL_ADC_Start_IT(&hadc1);
244     /* USER CODE END 2 */
245
246     /* Infinite loop */
247     /* USER CODE BEGIN WHILE */
248     while (1)
```

Proteus Schema:



بخش امتیازی:

این تکه کد ها در این بخش اضافه شده اند:

```
volatile int pressed_buttons_cnt = 0; // counter
volatile int two_buttons_pressed = 0; // flag
```

```
88 /* Private user code -----*/
89 /* USER CODE BEGIN 0 */
90 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
91 {
92     if ((htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_3) || (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
93     {
94         if (Is_First_Captured == 0) // if the first rising edge is not captured
95         {
96             IC_Val1 = HAL_TIM_ReadCapturedValue(htim, ((htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) ? TIM_CHANNEL_2 : TIM_CHANNEL_3)); // read the first value
97             Is_First_Captured = 1; // set the first captured as true
98             pressed_buttons_cnt++;
99             if(pressed_buttons_cnt == 2) {
100                 two_buttons_pressed = 1;
101             }
102         }
103         else if(two_buttons_pressed == 0) // If the first rising edge is captured, now we will capture the second edge
104         {
105             IC_Val2 = HAL_TIM_ReadCapturedValue(htim, ((htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) ? TIM_CHANNEL_2 : TIM_CHANNEL_3)); // read second value
106             if (IC_Val2 > IC_Val1)
107             {
108                 Difference = IC_Val2 - IC_Val1;
109             }
110             else if (IC_Val1 > IC_Val2)
111             {
112                 Difference = ((htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) ? PeriodTIM5 : PeriodTIM3) - IC_Val1 + IC_Val2;
113             }
114             if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2)
115             {
116                 if (!(N == 1 && (Difference >= threshold)))
117                     N += ((Difference < threshold) ? +1:-1);
118             }
119             if (adc_value != -1) {
120                 htim2.Instance->PSC = adc_value/2;
121                 HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
122                 adc_delay_mode = (Difference < threshold) ? 1:2;
123                 buzzer_flag = 1;
124             }
125         }
126     }
127 }
```

```
127         buzzer_flag = 1;
128     }
129 }
130 else
131 {
132     if (!(M == N && (Difference < threshold)) || (M == 1 && (Difference >= threshold))))
133         M += ((Difference < threshold) ? +1:-1);
134     htim2.Instance->PSC = adc_value;
135     HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
136     adc_delay_mode = (Difference < threshold) ? 1:2;
137     buzzer_flag = 1;
138 }
139 }
140
141 __HAL_TIM_SET_COUNTER(htim, 0); // reset the counter
142 Is_First_Captured = 0; // set it back to false
143 }
144 else
145 {
146     pressed_buttons_cnt--;
147     if(pressed_buttons_cnt == 0) {
148         two_buttons_pressed = 0; // set flag FALSE
149     }
150     __HAL_TIM_SET_COUNTER(htim, 0); // reset the counter
151     Is_First_Captured = 0;
152 }
153 }
154 }
```

```

157 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
158 {
159     if (htim->Instance == TIM4)
160     {
161         if (buzzer_flag == 1) {
162             ctr +=1;
163             if (ctr == adc_delay_mode * 2e5) // TODO: 2e5
164             {
165                 HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_2);
166                 ctr = 0;
167             }
168         }
169
170         adc_ctr += 1;
171         if(adc_ctr == 1e3) { // TODO: 1e5
172             adc_value = readADC();
173             htim2.Instance->PSC = adc_value;
174             adc_ctr = 0;
175         }
176
177         if(two_buttons_pressed == 1) {
178             int buzzer_freq = cal_buzzer_freq();
179             lcd_show_number((buzzer_freq < 1 ? 0:buzzer_freq)%10000);
180         }
181         else {
182             lcd_show_number(pascal(N, M));
183         }
184     }
185 }

```

نمونه خروجی:

